

Document Number: P0997

Topic: Retire Pernicious Language Constructs in Module Contexts

Date: 2018-10-09

Reply-to: Nathan Myers ncm@cantrip.org

Authors: Nathan Myers (MayStreet), Adam Martin (MongoDB), Eric Keane (Intel)

Audience: EWG

Retire Pernicious Language Constructs in Module Contexts

This paper presents a collection of small, independent proposals to improve C++ by making pernicious constructs in the language ill-formed when parsed in module context. Criteria for inclusion in the list include:

- A recommended alternative is well-known that is objectively better in all cases, and can be suggested in a compiler error message.
- The pernicious construct becomes ill-formed, so that any code that compiles in both module and non-module contexts has identical meaning.
- Where the construct exists in standard C++XX, the alternative may be used under the same standard C++XX. I.e., adopting the suggested alternative does not require upgrading to a newer compiler.

We assume some users will need to move code into and out of module context, without change. In particular, a file might have `#ifdef` header and footer blocks allowing the same file to be used different contexts, perhaps even simultaneously in different builds.

It also lists some other proposals, listed separately, that should be considered conditional on any of the primary proposals being accepted.

Introduction

The advent of modules presents a once-in-a-generation opportunity to leave behind language constructs that cause continuing problems for all users. The language, without the chosen constructs, would be objectively better by any reasonable measure. The module context already defines a language subset; adopting any of the proposals presented does not define a new subset; it only refines the subset implied by module context.

Our expectation is that at least some implementations will offer, as transition aids, an option (e.g. `"-faccept-retired"`) to accept the retired construct without complaint, even in module context, and an option (e.g. `"-Wretired"`) to issue a warning for uses of the constructs outside module contexts.

An example of an appealing idea that *violates* the criteria, and will not be suggested here, would be to make `NULL`, and implicit conversion of literal `0` to `τ*`, ill-formed. It is rejected because users of C++2003 cannot use the recommended alternative `nullptr`.

In addition, further below, we have proposals that would make sense to entertain if any of the proposed construct restrictions are adopted.

For each proposal that is accepted, we will solicit guidance on whether to also deprecate the construct where it is still well-formed.

Proposals

1. Most Vexing Parse

Every declaration described by "most vexing parse" can be expressed clearly, unambiguously, and with less text. [It has been said that it is already ill-formed in N4727. Can someone please identify where that may be checked?]

Proposal: Any construct that evokes the "most vexing parse" is ill-formed in module context.

2. Mixed-signedness ordered comparisons

Mixed-signedness ordered comparisons -- ops<, >, etc. -- already generate warnings in most compilers. Someday we hope we can make them produce the right answers, but the first step in that direction is to forbid them in C++20, at least in modules.

The suggested alternative is to cast one of the arguments, or use a library function such as is proposed in p0586.

Proposal: mixed-signed-integral comparisons are ill-formed in module context.

3. Implicit narrowing integer conversions

Where intended, narrowing conversions may be specified clearly with a cast, preceded by a range check where appropriate.

Proposal: narrowing integer conversions are ill-formed in module context.

4. NULL is nullptr

We got `nullptr` in C++11, but a great deal of code is still obliged to use `NULL`. That does not mean we must continue to permit its use in non-pointer expressions, in module context. Code that needs a literal zero can use literal zero.

Proposal: In module context, `NULL` is a keyword that aliases `nullptr`. The lexeme is not matched by any macro `NULL` that may be defined before the module, and cannot be `#defined` in a module. Use in a context that assumes it is a literal zero is ill-formed, as is use anywhere that use of `nullptr` would not be well-formed.

5. Disallow C-style casts via functional notation

The functional notation type conversion `T(v)` [`expr.type.conv`] looks syntactically like a constructor call, but is actually equivalent to a C-style cast `(T)v`. This is especially error-prone in (variadic) templates such as

```
template<class T, class... A> T construct(A... a) { return T(a...); }  
void* p = construct<void*>(5);
```

where `reinterpret_cast<void*>(5)` is obviously not the intent.

Proposal: Condition the case

If the initializer is a parenthesized single expression, the type conversion expression is equivalent to the corresponding cast expression (8.5.3).

from [`expr.type.conv`] on non-module context. When a cast is required, it can be written explicitly as `static_cast<T>(v)`, `const_cast<T>(v)`, `reinterpret_cast<T>(v)`, or even `(T)v`, instead of `T(v)`.

(Note: this affects [dcl.list.init]/3.8, where `T(v)` needs to be replaced with `static_cast<T>(v)`.)

Conditional Proposals

The following proposals should be considered if any of the above is adopted.

1. extern "module-ready C++"

If any of the above proposals is adopted, many users will wish for a way to apply the restriction also to their non-module code, as a transition aid. Such a scope might be denoted by enclosure in an extern "... " { . . . } block, after the manner of extern "C", or by an attribute applied to such a scope, e.g. extern "C++" [[...]] { . . . }. The word "strict" itself has unfortunate resonance with usage in generally permissive languages. A precise choice might be "module-ready".

Other Proposals

We accumulate other ideas here that, while they do not strictly satisfy the criteria for inclusion above, we think deserve consideration on their own merits.

1. spaceship comparison of signed and unsigned value

Mixed-sign comparisons using op< etc. have often-surprising results in C++17 and earlier. Those are to be ill-formed in some future standard, including in module context. In the WP, operator<=> applied to integers with different signedness is ill-formed. We think <=> should produce a useful result for this case, where possible and consistent with ops< etc. In practice this probably requires ops< etc. for this case to be ill-formed in module context in C++20.

Proposal: Mixed-signed ordered comparison with op<=>, in module context, produces arithmetically correct results, e.g. -1 <=> 0u reports "less than".

Conclusion

We solicit suggestions for other language warts that could be left behind as we move our code into modules, and for other adaptations to a world with reasoned restrictions on code in modules. Fertile sources of problem constructs are compiler warning-message lists and coding guidelines. Unfortunately, most items in any such list cannot be fixed here, but each is likely to contain a few viable items not found elsewhere.

Our request to the EWG is to vote on each of the proposals individually, with no implication of outcome for one item on others. We leave it up to EWG to maintain whatever consistency is appropriate.